

Master DSBD & IA - Module Algorithmes d'Optimisation

Projet 13 - Option 4: Robustesse des GNN face au bruit dans le graphe

FINAL REPORT

Optimization of Graph Neural Networks under Graph Perturbations

Adam vs AdamW vs RMSProp vs AdaGrad vs SGD on the Cora Citation Network

Main result

Adam is the most reliable optimizer in the fixed protocol: 79.9% clean accuracy and 70.2% mean accuracy across all clean and perturbed experiment rows.

Field	Value
Dataset	Cora Citation Network
Model	2-layer Graph Convolutional Network
Optimizers	Adam, AdamW, RMSProp, AdaGrad, SGD
Perturbations	Feature noise, edge removal, fake edge addition
Prepared by	Oussama Ashad and Mouhcine
Date de remise	20 juin 2026
Repository	github.com/Maaskk/gnn-optimization-under-graph-perturbations-cora
Dashboard	maaskk.github.io/gnn-optimization-under-graph-perturbations-cora/

Table des matieres / Table of Contents

#	Section	Focus
1	Abstract and key findings	Report
2	Introduction	Report
3	Related work	Report
4	Methodology	Report
5	Results	Report
6	Discussion	Report
7	Limitations and validity	Report
8	Conclusion and recommendations	Report
9	Bibliography	Report
10	Annexes	Report

The contents page is static so it renders correctly in headless PDF/PNG export. The document also uses real Word heading styles for navigation.

1. Abstract and key findings

This project studies how optimizer choice affects the stability and robustness of a Graph Convolutional Network trained on the Cora citation network. Five optimizers are compared under a fixed protocol: Adam, AdamW, RMSProp, AdaGrad, and SGD. The same GCN architecture, data split, seed, learning rate, weight decay, dropout, and epoch budget are used for every run.

Answer to the research question

Adam is the best overall choice for this fixed GCN/Cora protocol. It ties AdamW on the clean graph, leads the aggregate ranking across all 65 runs, and remains strongest under edge removal and fake edge addition.

Finding	Result	Evidence
Best clean accuracy	Adam / AdamW	0.799
Best aggregate accuracy	Adam	0.702
Best aggregate macro F1	Adam	0.694
Hardest perturbation	Feature noise	All adaptive optimizers fall near 0.52-0.54 at 30% noise
Weakest optimizer in fixed protocol	SGD	Near random-guess baseline

2. Introduction

Graph Neural Networks learn from both node attributes and graph topology. In citation networks such as Cora, each paper is represented as a node, citation links define graph edges, and bag-of-words features describe document content. Because GCN message passing aggregates information through neighborhoods, graph perturbations can affect both convergence and final classification quality.

The objective is to compare optimizer behavior, not to tune each optimizer to its own best possible configuration. This is why all experiments use a common fixed protocol. The central question is: which optimizer best preserves node-classification performance when features or edges are corrupted?

3. Related work

Kipf and Welling introduced the two-layer GCN model for semi-supervised classification on citation networks and reported strong Cora performance with a small labeled training set [1]. GraphSAGE and GAT later expanded the GNN family through inductive aggregation and attention mechanisms [2, 3].

For optimization, Adam combines momentum with adaptive per-parameter learning rates [4], AdamW decouples weight decay from Adam's adaptive update [5], RMSProp uses a moving average of squared

gradients, AdaGrad accumulates squared gradients, and SGD provides the non-adaptive baseline. Prior robustness work shows that GNNs are sensitive to structure and feature perturbations [7, 8].

4. Methodology

4.1 Dataset

Property	Value	Note
Nodes	2,708	Citation graph nodes
Directed edges	10,556	Cora edge_index entries
Node features	1,433	Bag-of-words feature vector
Classes	7	Paper topic labels
Training nodes	140	Planetoid split
Validation nodes	500	Planetoid split
Test nodes	1,000	Planetoid split

4.2 Model and fixed protocol

Parameter	Value	Detail
Model	2-layer GCN	PyTorch Geometric
Hidden channels	16	GCNConv(1433 -> 16 -> 7)
Dropout	0.5	Training only
Learning rate	0.01	Same for all optimizers
Weight decay	0.0005	Same for all optimizers
Epochs	200	Fixed budget
Seed	42	All experiments
Perturbation severity	5%, 10%, 20%, 30%	Per perturbation type

4.3 Perturbation design

Three perturbation families are tested. Feature noise adds Gaussian noise $N(0, \text{severity})$ to node features. Edge removal deletes a random fraction of citation edges. Fake edge addition injects random non-existing citation links. Each optimizer is evaluated on one clean condition plus twelve perturbed conditions, for 65 total experiment rows.

5. Results

5.1 Accuracy benchmark: is 79.9% too low?

No. A clean accuracy of 79.9% is reasonable for a simple two-layer GCN under a single fixed optimizer-comparison protocol. The original GCN paper reports 81.5% on Cora [1], but that number is not directly identical to this experiment because it comes from its own setup, averaging, and training choices. Our goal is a fair optimizer comparison under one fixed protocol, not maximum leaderboard tuning.

Interpretation

The 70.2% aggregate accuracy is lower because it averages clean and perturbed graphs. It should not be compared directly with a clean-only Cora benchmark.

5.2 Clean graph performance

Optimizer	Accuracy	Macro F1	Final loss	Time
Adam	0.799	0.790	0.316	12.1s
AdamW	0.799	0.790	0.058	21.1s
RMSProp	0.795	0.787	0.381	41.7s
AdaGrad	0.719	0.719	1.411	24.5s
SGD	0.148	0.056	1.944	19.4s

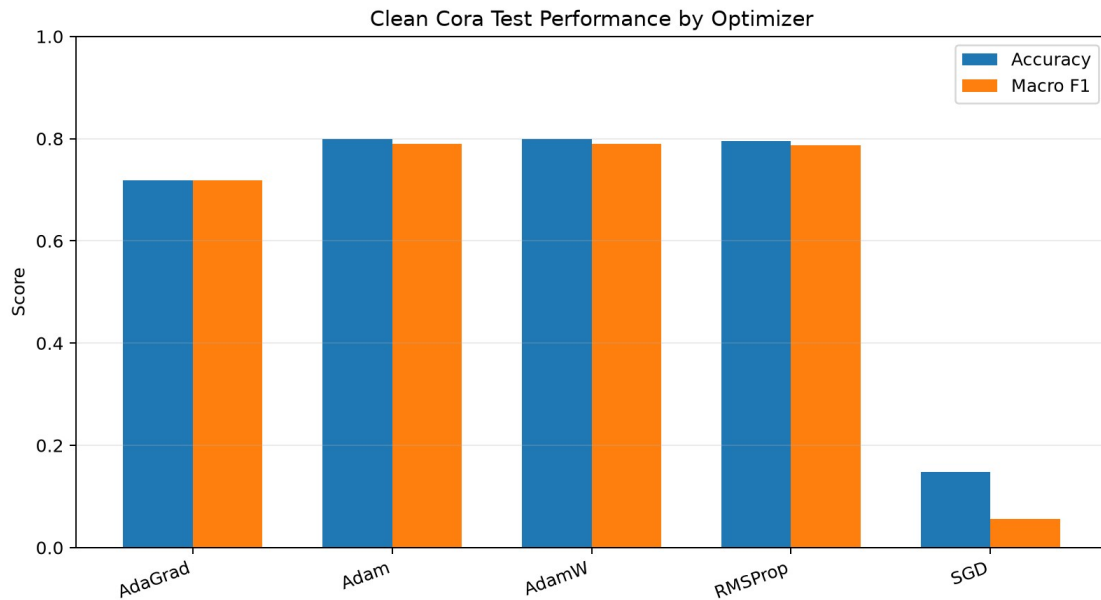


Figure 1 - Clean Cora test accuracy and macro F1-score.

Adam and AdamW tie at 0.799 clean test accuracy. RMSProp follows very closely at 0.795. AdaGrad learns but remains lower, while SGD stays near random-guess behavior under the fixed learning rate.

5.3 Aggregate performance

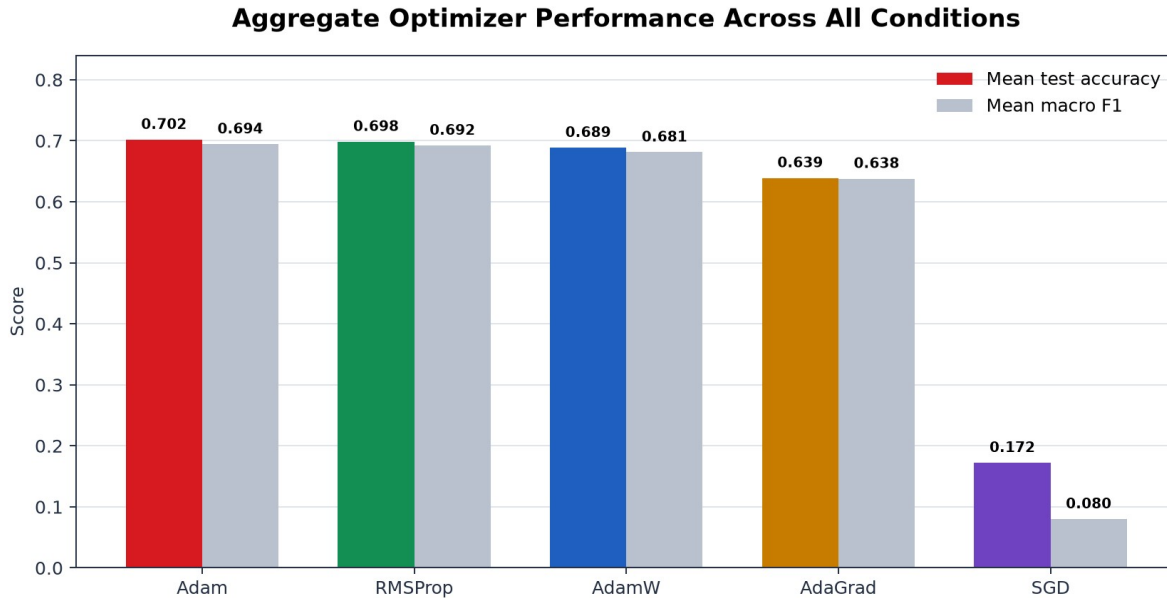


Figure 2 - Aggregate mean accuracy and macro F1 across all 65 runs.

Optimizer	Clean	Edge removal	Fake edges	Feature noise
Adam	0.799	0.783	0.754	0.543
RMSProp	0.795	0.781	0.751	0.539
AdamW	0.799	0.781	0.728	0.529
AdaGrad	0.719	0.714	0.649	0.534
SGD	0.148	0.141	0.202	0.180

5.4 Robustness by perturbation type

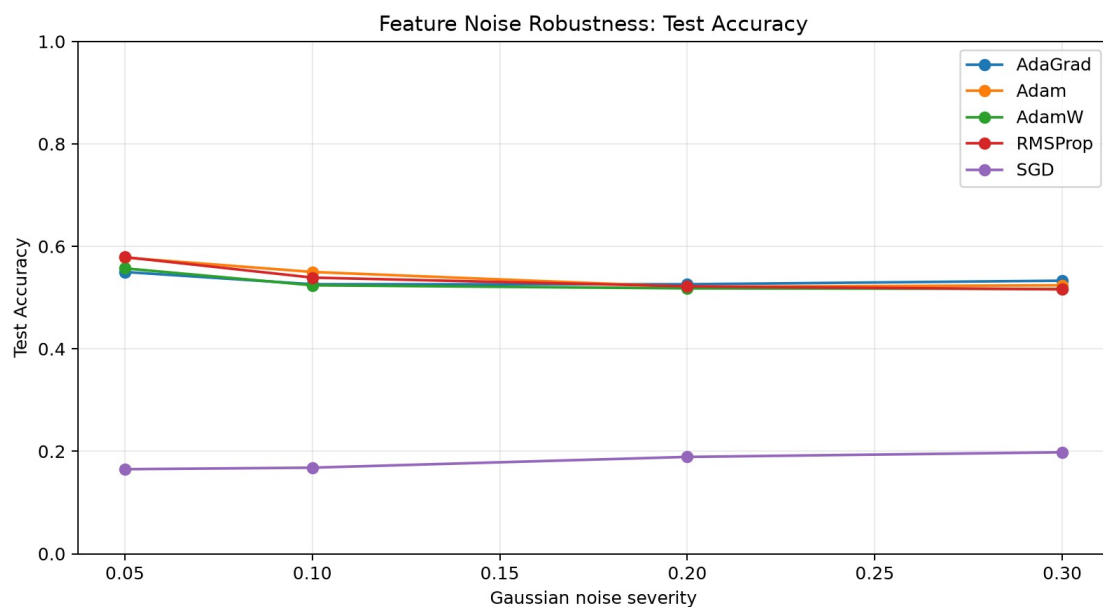


Figure 3 - Feature noise causes the largest accuracy collapse.

Feature noise is the hardest condition: by 30% severity, Adam, AdamW, RMSProp, and AdaGrad all converge around the 0.516-0.533 accuracy band. This suggests the disturbance challenges the feature representation and the two-layer GCN itself, not only the optimizer.

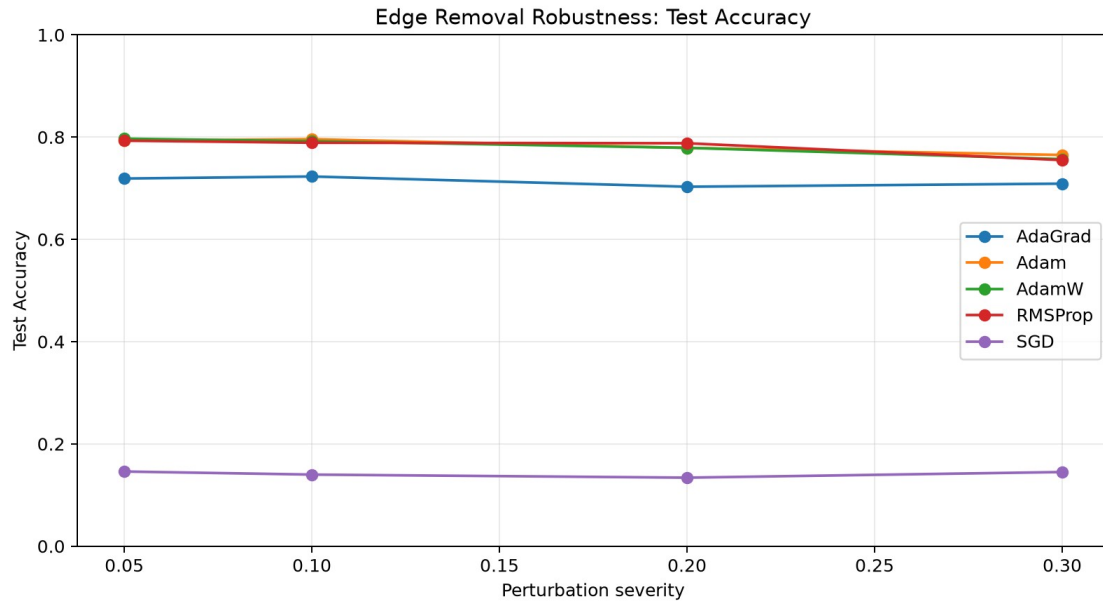


Figure 4 - Edge removal is tolerated best by adaptive optimizers.

Edge removal produces the smallest drop. Adam, AdamW, and RMSProp remain close to their clean scores even at 30% edge deletion, showing that the Cora graph retains enough redundant neighborhood information within this severity range.

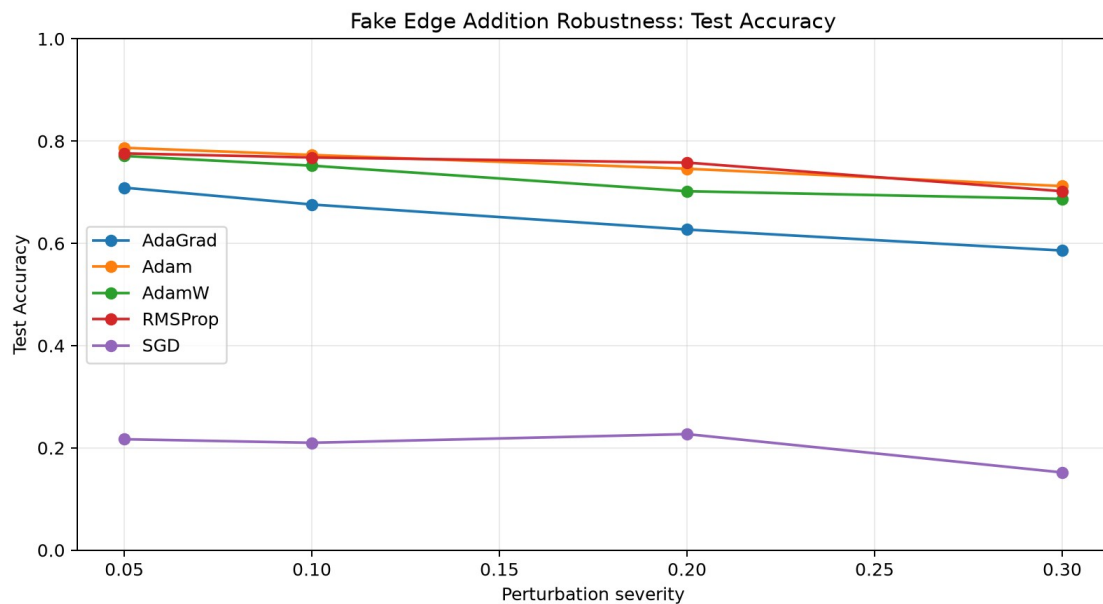


Figure 5 - Fake edge addition causes a monotonic structural degradation.

Fake edges are more damaging than edge removal because they add misleading neighborhoods. Adam has the smallest degradation among the adaptive optimizers, while AdaGrad falls more sharply under injected links.

5.5 Training dynamics

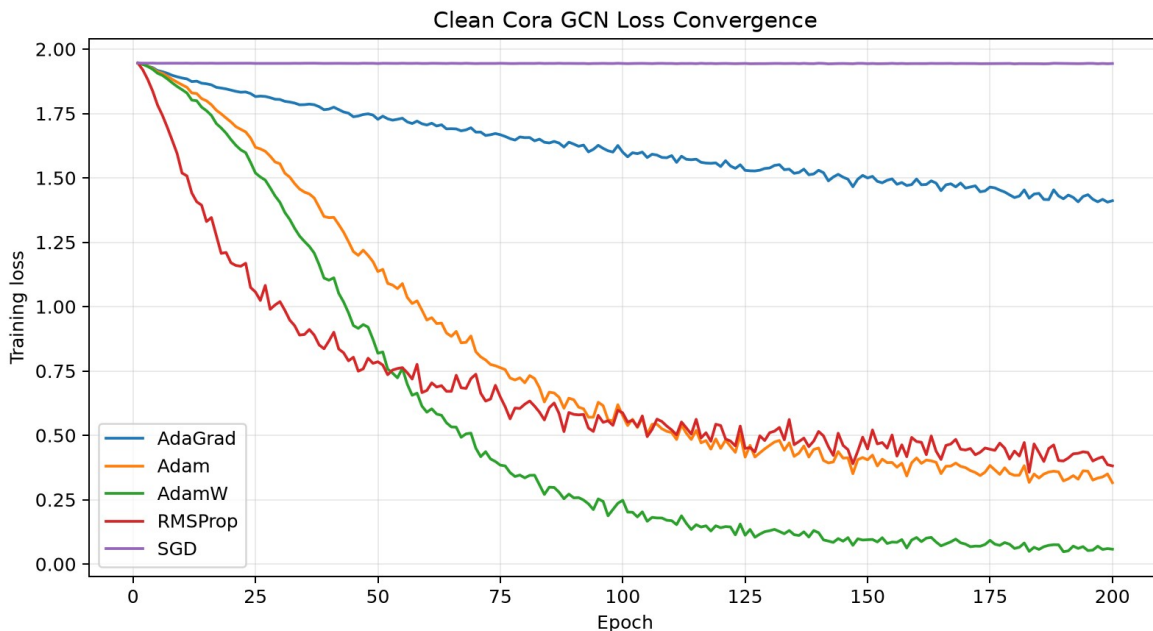


Figure 6 - Training and validation loss over 200 epochs on the clean graph.

AdamW reaches the lowest training loss, but the wider train/validation gap indicates tighter fitting to the small 140-node training set. RMSProp reaches a strong validation plateau quickly, while SGD does not escape its near-random initial behavior.

5.6 Accuracy drop matrix

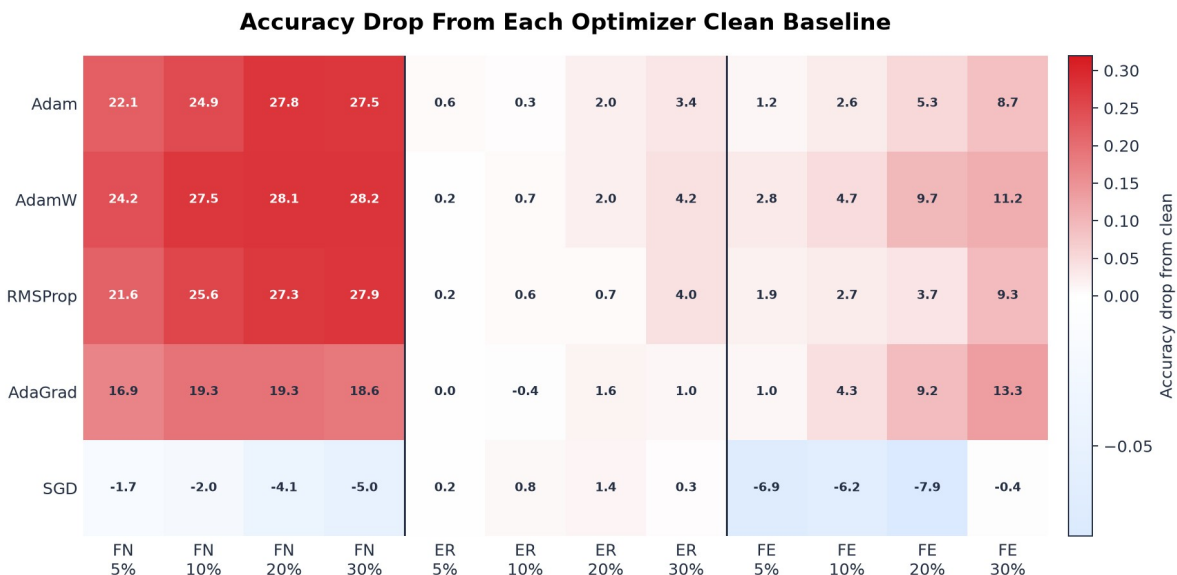


Figure 7 - Accuracy drop from each optimizer's own clean baseline.

The heatmap makes the robustness pattern explicit. Feature noise generates the largest red block across adaptive optimizers. Edge removal stays pale, indicating small losses. Fake edges form an intermediate degradation pattern that grows with severity.

6. Discussion

Adam is the most balanced optimizer in this study. It does not always converge fastest, but it combines strong clean accuracy, best aggregate robustness, and reliable behavior under structural corruption. RMSProp is a strong second and is especially attractive when fast convergence to a stable plateau matters.

AdamW matches Adam on the clean graph, but its lower training loss does not translate into better robustness. In this fixed protocol, AdamW falls behind Adam under fake edge addition and has a wider train/validation loss gap. AdaGrad is conservative and stable under edge removal but has a lower accuracy ceiling. SGD is included as a baseline and demonstrates why adaptive optimization is important for this GCN/Cora setup.

7. Limitations and validity

Limitation	Meaning
Single seed	All experiments use seed 42; multiple seeds would give confidence intervals.
Fixed learning rate	The protocol is fair for comparison but may disadvantage SGD, which often needs tuning or momentum.
One dataset	Cora is standard but small; PubMed and CiteSeer would test scale and transfer.
One architecture	The conclusions apply to this 2-layer GCN, not necessarily GAT or GraphSAGE.
Random perturbations	The perturbations are random, not adversarial attacks.

These limitations do not invalidate the project. They define the interpretation: this is a controlled optimizer comparison under graph noise, not a universal claim about every GNN or every citation dataset.

8. Conclusion and recommendations

The project concludes that Adam is the recommended optimizer for this GCN on Cora when robustness under graph perturbations is part of the evaluation. RMSProp is a close alternative, especially for rapid convergence. AdamW is excellent on clean Cora but less robust to fake-edge structural noise in this run. AdaGrad is cautious and stable but generally lower. SGD performs poorly under the fixed protocol.

Final recommendation

Use Adam as the default optimizer for the final project demonstration, and explain that its advantage is not only clean accuracy but also stable performance across feature and structural perturbations.

9. Bibliography

- [1] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR 2017. arxiv.org/abs/1609.02907
- [2] Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. NeurIPS 2017. arxiv.org/abs/1706.02216
- [3] Velickovic, P., et al. (2018). Graph Attention Networks. ICLR 2018. arxiv.org/abs/1710.10903
- [4] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR 2015. arxiv.org/abs/1412.6980
- [5] Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019. arxiv.org/abs/1711.05101
- [6] Hinton, G., Srivastava, N., & Swersky, K. (2012). RMSProp. Neural Networks for Machine Learning, Lecture 6e.
- [7] Zugner, D., Akbarnejad, A., & Gunnemann, S. (2018). Adversarial Attacks on Neural Networks for Graph Data. KDD 2018. arxiv.org/abs/1805.07984
- [8] Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR Workshop. arxiv.org/abs/1903.02428

10. Annexes

Artifact	Location
Repository	https://github.com/Maaskk/gnn-optimization-under-graph-perturbations-cora
Interactive dashboard	https://maaskk.github.io/gnn-optimization-under-graph-perturbations-cora/
Full result table	results/all_results.csv
Clean results	results/clean_optimizer_results.csv
Feature noise results	results/feature_noise_results.csv
Edge removal results	results/edge_removal_results.csv
Fake edge results	results/fake_edge_addition_results.csv
Loss history	results/clean_loss_history.csv
Frontend source	docs/index.html, docs/assets/app.js, docs/assets/styles.css